

Innovate Solutions

ROOMTRACKER

semana 18

HumanServices by:
Juan Felipe Gutierrez Adarme
Daniel Galvis Betancourth



RoomTracker

1. EL PROBLEMA



Dificultad para ubicarse en el campus



Falta de rutas internas claras



Dependencia de terceros para orientación

2. NUESTRA SOLUCIÓN



Mapa interactivo con rutas GPS dentro del campus



Información detallada de edificios y servicios



Indicaciones claras y navegación intuitiva

3. PROPUESTA DE VALOR



Mapa interactivo

Explora el campus con mapas detallados e interactivos.

Orientación precisa y contextualizada dentro del campus



Servicios detallados

Accede a información completa de espacios, horarios y servicios.



Navegación contextualizada

Rutas personalizadas según tu ubicación, contexto y necesidades.

4. VENTAJAS COMPETITIVAS



Accesibilidad

Diseñado para todos los estudiantes.



Precisión

Tecnología GPS para ubicaciones exactas.



Contextualidad

Información relevante según tu contexto.



Confiabilidad

Datos actualizados y verificados constantemente.



Escalabilidad

Crece junto a la universidad y sus necesidades.



Adaptabilidad

Se ajusta a distintos campus y comunidades.

5. MODELO DE NEGOCIO

CANALES



B2B Universidades

Alianzas estratégicas con universidades e instituciones educativas.

ESTRUCTURA DE COSTES



Desarrollo
Investigación, UX/UI, Desarrollo de la app.



Backend
Infraestructura en la nube, bases de datos, APIs y servicios.



Mantenimiento
Actualizaciones, soporte técnico y mejoras continuas.

MÉTRICAS



✓ **Usuarios Activos**
Crecimiento continuo de la comunidad.

✓ **Rutas Completadas**
Medición del uso y eficacia de la navegación.

✓ **Adopción**
Tasa de adopción por universidades y estudiantes.



RoomTracker

Encuentra tu lugar. Enfócate en lo importante.

Historias de usuario



ALMACENAMIENTO LOCAL - Android Device



A) Room Database (SQLite)

Cache local y datos offline

	UsuariosLocal	id, nombre, email, fotoPerfil
	MisAmigos	idAmigo, nombre, estado
	HistorialChat	idMensaje, contenido, timestamp
	HistorialActividad	fecha, pasos, distancia
	CacheEdificios	idEdificio, nombre, detalles

~50-100 MB



B) SharedPreferences

Configuración y tokens

	carpeta_mapas (institución actual)
	auth_token (JWT token)
	fcm_token (Firebase Cloud Messaging)
	preferencias_usuario (idioma, tema)
	último_sincronizado (timestamps)



C) Files Directory

Caché de datos descargados

	edificios.json
	pois.json
	graph.json
	campus_updated.geojson
	edge_geometry.json

~10-20 MB

Sincronización + Caché



FIREBASE - Servicios en Tiempo Real



A) Firebase Cloud Messaging (FCM)

Notificaciones push



Casos de uso:

- Mensajes nuevos
- Notificaciones de eventos
- Alertas de cambios en datos



B) Firebase Performance Monitoring (Optional)

Monitoreo de rendimiento

Métricas: Tiempo de respuesta, crashes

Escucha notificaciones

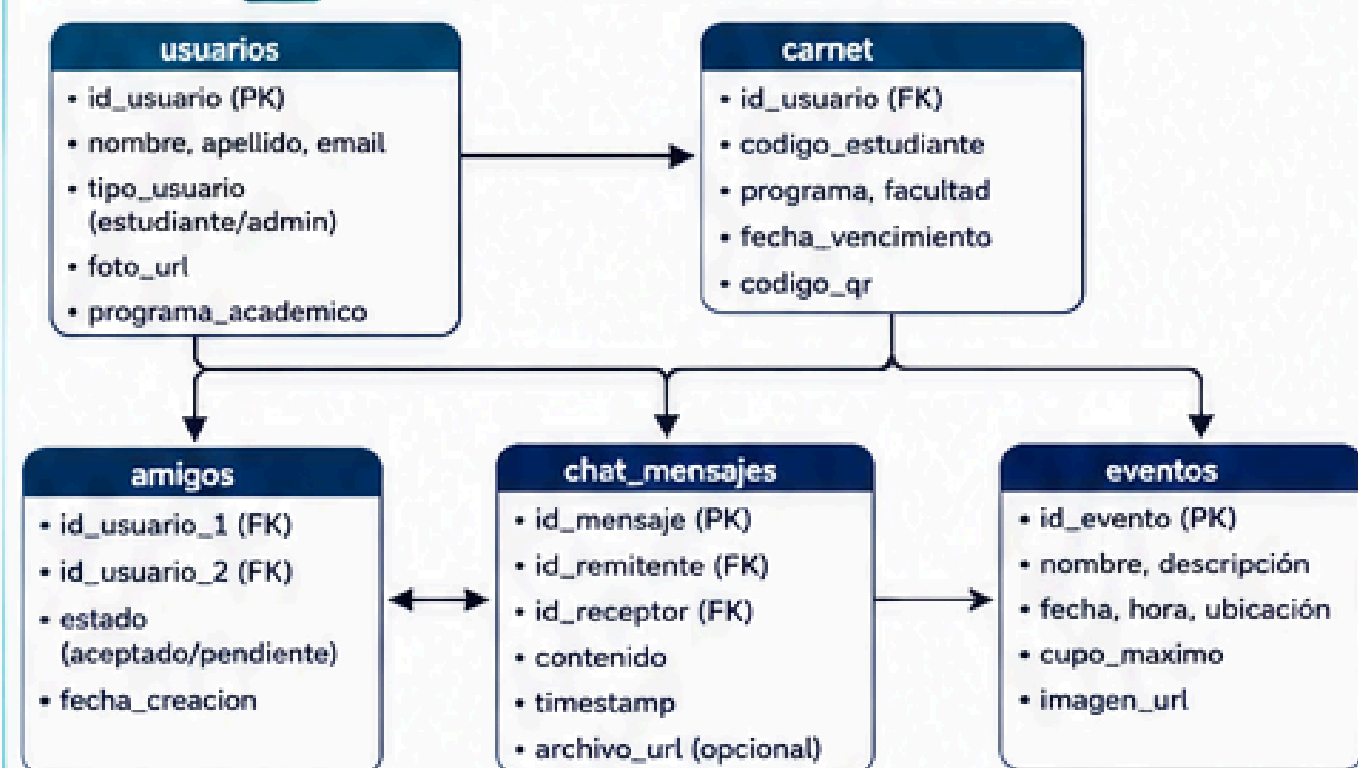
Envía métricas



SUPABASE - Backend Principal & Storage



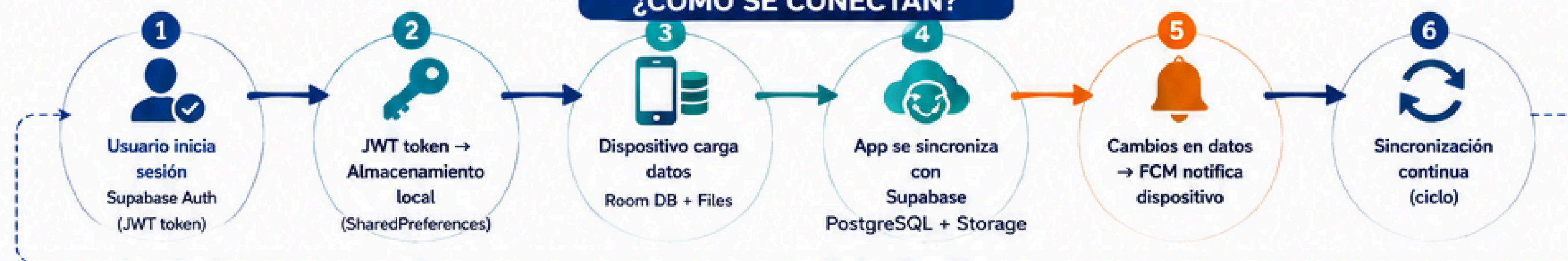
PostgreSQL Relational DB



Cloud Storage Buckets

	1. avatars/	Location: javeriana/{uid}.jpg Size: ~1-2 MB por usuario Contenido: Fotos de perfil de usuario		PÚBLICO (downloadPublic)
	2. mapas/	Location: {carpeta_institucion}/ Contenido: edificios.json, pois.json, graph.json, edge_geometry.json, campus_updated.geojson Size: ~5-10 MB por institución		PÚBLICO (downloadPublic)
	3. chat-media/	Location: javeriana/{chatId}/ Contenido: Imágenes, documentos enviados en chat Size: Variable (dependiente del usuario)		PRIVADO (auth requerido)
	4. notas/	Location: {carpeta_institucion}/{uid}.json Contenido: Estadísticas académicas JSON Size: ~10-50 KB por usuario Schema: {idUsuario, nombre, apellido, promedio_acumulado, periodos[]}		PRIVADO (RLS policy)
	5. distancias/	Location: javeriana/{uid}.json Contenido: Historial de pasos/actividad Size: ~5-10 KB por usuario Schema: {idUsuario, totalPasos, totalDistanciaMetros, historial[]}		PRIVADO (RLS policy)

¿CÓMO SE CONECTAN?



SINCRONIZACIÓN Y FLUJOS DE DATOS



1 Autenticación

- 1 Usuario ingresa credenciales
- 2 Envía a Supabase.auth.signInWithPassword()
- 3 Recibe JWT token + user ID
- 4 Guarda token en SharedPreferences
- 5 Descarga datos de usuario a Room DB



2 Lectura de Datos

- 1 App solicita datos (ej: edificios)
- 2 Verifica si existe en caché local (Room/Files)
- 3 Si existe: usa caché (offline-first)
- 4 Si no existe: consulta Supabase PostgreSQL/Storage
- 5 Guarda en caché local y muestra datos



3 Escritura de Datos

- 1 Usuario realiza cambios (ej: nuevo mensaje)
- 2 Guarda localmente (Room DB) para respuesta inmediata
- 3 App sincroniza con Supabase PostgreSQL/Storage
- 4 Confirma éxito y actualiza caché local
- 5 FCM notifica otros dispositivos (si aplica)



Almacenamiento Local (Android)



Firebase (Tiempo Real)



Supabase (Backend & Storage)

Flujo de Datos

Sincronización Continua

Público

Privado (Autenticado)



RoomTracker: Servicios REST Externos

Integración de APIs y Consumo de Servicios

Arquitectura de Conectividad con Servicios Externos





Supabase: La Base de Datos Principal de RoomTracker

PostgreSQL + Auth + Storage + Realtime + API REST



1 ARQUITECTURA DE SUPABASE



1 PostgreSQL Database

Base de datos relacional principal

PostgreSQL 14+

- Tablas relacionales
- Triggers y funciones
- JSON/JSONB fields
- Constraints (FK, PK, UNIQUE)
- Full-text search
- Transacciones ACID



2 Supabase Auth

Autenticación y manejo de sesiones

Basado en PostgreSQL `auth.users` table

- Sign up/Sign in
- Email verification
- OAuth (Google, GitHub, etc.)
- JWT tokens
- Password reset
- MFA support
- Refresh tokens



3 Supabase Storage

Almacenamiento de archivos

S3-compatible

- Múltiples buckets
- Resumable uploads
- Automatic compression
- Public/Private access
- CDN integration
- RLS policies



4 PostgREST API

API REST auto-generada (PostgREST)

- CRUD operations
- Relationships
- Aggregations
- Filtering y sorting
- Full-text search



5 Realtime

Suscripciones en tiempo real (opcional)

- Database change notifications
- Presence tracking
- Broadcast messages

2 ESQUEMA POSTGRESQL COMPLETO

Base de datos: `roomtracker_db`

1 auth.users (Built-in)

auth schema (Supabase managed)

- `id`: UUID (PK) - Unique user identifier
- `email`: VARCHAR - User email
- `encrypted_password`: VARCHAR - Hashed password
- `email_confirmed_at`: TIMESTAMP
- `last_sign_in_at`: TIMESTAMP
- `aud`: VARCHAR - Audience claim
- `confirmation_token`: VARCHAR
- `recovery_token`: VARCHAR
- `email_change`: VARCHAR
- `phone`: VARCHAR (opcional)
- `confirmed_at`: TIMESTAMP
- `created_at`: TIMESTAMP
- `updated_at`: TIMESTAMP

Relación: 1 → ∞ con `public.usuarios`

Índices: `email` (UNIQUE), `id` (PK)

RLS: Gestionado por Supabase Auth

5 public.chat_mensajes

Mensajes entre usuarios

- `id_mensaje`: UUID (PK)
- `id_remitente`: UUID (FK → `usuarios.id_usuario`)
- `id_receptor`: UUID (FK → `usuarios.id_usuario`)
- `contenido`: TEXT (NOT NULL)
- `archivo_url`: TEXT (opcional)
- `leido`: BOOLEAN DEFAULT false
- `created_at`: TIMESTAMP
- `updated_at`: TIMESTAMP

Relaciones: → FK a `usuarios(id_remitente)`
→ FK a `usuarios(id_receptor)`

Índices: (`id_remitente`, `id_receptor`, `created_at`), `leido`

RLS: Usuarios pueden ver sus conversaciones

2 public.usuarios

Perfil de usuario y datos académicos

- `id_usuario`: UUID (PK, FK → `auth.users.id`)
- `email`: VARCHAR (UNIQUE)
- `nombre`: VARCHAR (NOT NULL)
- `apellido`: VARCHAR (NOT NULL)
- `tipo_usuario`: ENUM ('estudiante', 'profesor', 'admin')
- `programa_academico`: VARCHAR
- `facultad`: VARCHAR
- `foto_url`: TEXT - Avatar en Storage
- `fcm_token`: TEXT - Firebase token
- `carpeta_mapas`: VARCHAR
- `created_at`: TIMESTAMP DEFAULT now()
- `updated_at`: TIMESTAMP DEFAULT now()
- `deleted_at`: TIMESTAMP (soft delete)

Relaciones:

- PK → `auth.users(id)`
- 1:∞ `carnet(id_usuario)`
- 1:∞ `amigos(id_usuario_1, id_usuario_2)`
- 1:∞ `chat_mensajes(id_remitente, id_receptor)`
- 1:∞ `evento_participantes(id_usuario)`

Índices: `id_usuario` (PK), `email` (UNIQUE)

Trigger: `update_updated_at_on_usuarios`

RLS: SELECT/UPDATE propio o admin, DELETE solo admin

6 public.eventos

Eventos y actividades del campus

- `id_evento`: UUID (PK)
- `nombre`: VARCHAR (NOT NULL)
- `descripcion`: TEXT
- `fecha`: DATE (NOT NULL)
- `hora_inicio`: TIME (NOT NULL)
- `hora_fin`: TIME
- `ubicacion`: VARCHAR (NOT NULL)
- `cupo_maximo`: INTEGER
- `imagen_url`: TEXT
- `created_by`: UUID (FK → `usuarios.id_usuario`)
- `created_at`: TIMESTAMP
- `updated_at`: TIMESTAMP

Índices: `fecha`, `ubicacion`, `created_by`

RLS: SELECT publico, modificación solo admin/creador

4 public.carnet

Información del carnet universitario

- `id_carnet`: UUID (PK)
- `id_usuario`: UUID (FK → `usuarios.id_usuario`)
- `codigo_estudiante`: VARCHAR (UNIQUE, NOT NULL)
- `programa`: VARCHAR (NOT NULL)
- `facultad`: VARCHAR (NOT NULL)
- `fecha_vencimiento`: DATE
- `codigo_qr`: TEXT - QR code data
- `created_at`: TIMESTAMP
- `updated_at`: TIMESTAMP

Constraints:

- UNIQUE(`codigo_estudiante`)
- UNIQUE(`id_usuario`) - Un carnet por usuario
- CHECK (`fecha_vencimiento` > `created_at`)

Relacion: → FK a `usuarios(id_usuario)`

Índices: `id_usuario` (FK), `codigo_estudiante` (UNIQUE)

RLS: SELECT propio o admin, UPDATE solo admin

4 public.amigos

Relación de amistad entre usuarios

- `id_amistad`: UUID (PK)
- `id_usuario_1`: UUID (FK → `usuarios.id_usuario`)
- `id_usuario_2`: UUID (FK → `usuarios.id_usuario`)
- `estado`: ENUM ('pendiente', 'aceptado', 'bloqueado')
- `created_at`: TIMESTAMP
- `updated_at`: TIMESTAMP

Constraints:

- `id_usuario_1` < `id_usuario_2` (evitar duplicados)
- UNIQUE(`id_usuario_1`, `id_usuario_2`)
- CHECK (`id_usuario_1` != `id_usuario_2`)

Relaciones: → FK a `usuarios(id_usuario_1)`
→ FK a `usuarios(id_usuario_2)`

Índices: (`id_usuario_1`, `id_usuario_2`) UNIQUE, `estado`

RLS: Selección/Inserción/Actualización solo participantes

6 public.notas_academicas

Estadísticas académicas del usuario

- `id_nota`: UUID (PK)
- `id_usuario`: UUID (FK → `usuarios.id_usuario`)
- `periodo`: VARCHAR (NOT NULL)
- `promedio_acumulado`: NUMERIC (5,2)
- `materias_aprobadas`: INTEGER
- `total_creditos`: INTEGER
- `datos_json`: JSONB - Detalle de materias/periodos
- `created_at`: TIMESTAMP
- `updated_at`: TIMESTAMP

Índices: `id_usuario`, `periodo`

RLS: SELECT/UPDATE propio o admin

SEGURIDAD Y RLS (Row Level Security)

NIVEL DE SEGURIDAD	EJEMPLO DE POLÍTICA RLS
Row Level Security (RLS) en todas las tablas	Tabla: <code>usuarios</code>
Políticas basadas en <code>auth.uid()</code>	<pre>CREATE POLICY 'usuarios_select_policy' ON public.usuarios FOR SELECT USING (auth.uid() = id_usuario OR is_admin());</pre>
JWT tokens firmados	
Conexión HTTPS/TLS	
Validación de entrada	
Prevención de SQL Injection	
Backup automático diario	
Logs y auditoría	

REST API REST (PostgREST Auto-generada)

BASE URL: `https://[project].supabase.co/rest/v1`

Headers requeridos:

Authorization: Bearer `[jwt_token]`

apikey: `(anon_key o service_role)`

Content-Type: `application/json`

EJEMPLO DE CONSULTA

GET `/usuarios?id_usuario=eq.(uid)&select=id_usuario,nombre,apellido,email`

Respuesta:

```
{
  "id_usuario": "uuid", "nombre": "Juan", "apellido": "Pérez", "email": "juan@uni.edu"
}
```

RELACIONES PRINCIPALES



ALMACENAMIENTO (STORAGE BUCKETS)

BUCKET	USO	ACCESO	RLS
avatars	Fotos de perfil de usuarios	Público	Si
mapas	Mapas, edificios, POIs, grafos	Público	Si
chat-media	Archivos compartidos en chat	Privado	Si
notas	Estadísticas académicas (JSON)	Privado	Si
distancias	Historial de pasos/actividad	Privado	Si

URL base: `https://[project].supabase.co/storage/v1/object`

Acceso: Público (`/public/`) o Privado (requiere JWT token)

BENEFICIOS

- Backend como servicio (BaaS) completo
- Escalable y de alto rendimiento
- APIs automáticas y en tiempo real
- Seguridad empresarial con RLS

HERRAMIENTAS

- Supabase Dashboard
- SQL Editor
- Table Editor
- Logs Explorer
- Storage Browser
- API Docs (PostgREST)



INTEGRACIÓN CON ROOMTRACKER APP

La app Android consume la API REST (PostgREST) con JWT tokens para acceder a todos los datos de forma segura y eficiente.

INFORMACIÓN DEL PROYECTO

Región: `southamerica-east1` (São Paulo)

Versión Supabase: `Cloud (Managed)`

Base de datos: `roomtracker_db`



Documentación

Supabase

supabase.com/docs



RoomTracker: Arquitectura y Estructura del Proyecto

Estructura de Carpetas, Responsabilidades y Flujo de Datos

CONEXIONES EXTERNAS

SENSORES DEL DISPOSITIVO

- Step Detector**
(Sensor de pasos)
- GPS / Location Services**
(Ubicación)
- Cámara**
(Para foto de perfil)

SUPABASE - BASE DE DATOS EN LA NUBE

Base de datos PostgreSQL:

- Tabla "usuarios"**
ID, nombre, apellido, tipo, foto_url, session_token, fcm_token
- Tabla "carnet"**
código estudiante, programa, facultad, fecha vencimiento
- Tabla "institucion"**
ID institución, nombre, datos académicos

- Storage "avatars"**
Fotos de perfil de usuarios
- Storage "notas"**
Archivos JSON con estadísticas académicas por usuario
- Storage "mapas"**
Archivos geográficos (geojson, campus, puntos de interés)



Firestore Cloud Messaging (FCM)
Envío de notificaciones push en tiempo real

data/repository/

Responsabilidad:
ACCESO A DATOS
(intermediaria entre ViewModel y Supabase)

AuthRepository.kt

¿Qué hace?: Gestiona autenticación de usuario

Métodos principales:

- **signIn/signUp** → Conecta con Supabase Auth
- **uploadPhoto** → Sube foto a Storage "avatars"
- **downloadMapFiles** → Descarga mapas de Storage "mapas"
- **saveCarpetaMapas** → Guarda en SharedPreferences (memoria local)
- **clearMapFiles** → Limpia mapas descargados

Conexiones:

- Auth**
- Storage**
- SharedPrefs**

AcademicRepository.kt

¿Qué hace?: Gestiona datos académicos del usuario

Métodos principales:

- **loadEstadísticas** → Lee archivo JSON de Storage "notas"
- **saveEstadísticas** → Guarda cambios en JSON
- **loadCarnet** → Obtiene datos de tablas "carnet" y "usuarios"

Conexiones:

- Storage "notas"**
- BD "carnet"**
- BD "usuarios"**

PedometerRepository.kt

¿Qué hace?: Persiste datos de pasos y distancia

Métodos principales:

- **loadStats** → Descarga estadísticas de pasos previas
- **saveStats** → Guarda nuevo registro de pasos/distancia

Conexiones:

- Storage o BD**
(para historial de actividad)

viewmodel/

Responsabilidad:
LÓGICA Y ESTADOS
(procesa datos y reacciona)

AcademicStatsViewModel.kt

¿Qué hace?: Maneja estado de estadísticas académicas

Estados (StateFlow):

- statsState**
Loading | Success | Error
- saveState**
Idle | Saving | Saved | Error

Métodos principales:

- **loadStats()** → Llama a AcademicRepository.loadEstadísticas()
- **guardarMaterias()** → Modifica datos y llama a saveEstadísticas()

Flujo: Repository → ViewModel → UI

PedometerViewModel.kt ★ (GESTIONA SENSORES)

¿Qué hace?: Maneja conteo de pasos y detección de ubicación

Estados (StateFlow):

- stepsToday**
Pasos del día actual
- distanciaTotal**
Metros recorridos
- dentroCampus**
Si está dentro del campus
- historial**
Últimos 7 días

INTEGRACIÓN CON SENSORES:

- [SENSOR STEP DETECTOR]**
 - **stepListener** → Escucha cada paso
 - Solo cuenta si dentroCampus = true
 - Cada paso = +0.75 metros
 - Cada 50 pasos → **saveStats()** automático
- [GPS / LOCATION SERVICES]**
 - **locationCallback** → Recibe ubicación cada 10 segundos
 - Verifica si punto está dentro del campus (archivo geojson)
 - Actualiza dentroCampus (true/false)
- [ARCHIVO GEOJSON]**
 - **campus_updated.geojson** (descargado por AuthRepository)
 - Define polígono de campus
 - Usa algoritmo ray-casting para detectar si punto está dentro

Flujo: Sensores → ViewModel → Repository → BD

model/

Responsabilidad:
ESTRUCTURAS DE DATOS
(definiciones compartidas)

AcademicModels.kt

¿Qué hace?: Define modelos para académicas

Modelos principales:

- Materia.json** → código, nombre, nota, créditos
- Periodo.json** → año, ciclo, promedio, materias
- Estadísticas.json** → datos completos del estudiante
- CarnetData** → información del carnet

Función especial: recalcular()

- Recalcula promedios automáticamente cuando cambian materias
- Promedio = suma(nota x créditos) / créditos totales
- Se ejecuta: antes de guardar en BD

PedometerModels.kt (implícito)

¿Qué hace?: Define modelo de actividad

Modelos principales:

- DiaStats** → fecha, pasos, distancia
- Distancia.json** → ID usuario, total pasos, historial

Conexión:

Se serializa a JSON para guardar en BD



4 CAPA UI (Consumo)

Responsabilidad:
PRESENTACIÓN
(observa estados y muestra datos)

Las pantallas observan los StateFlow de los ViewModels

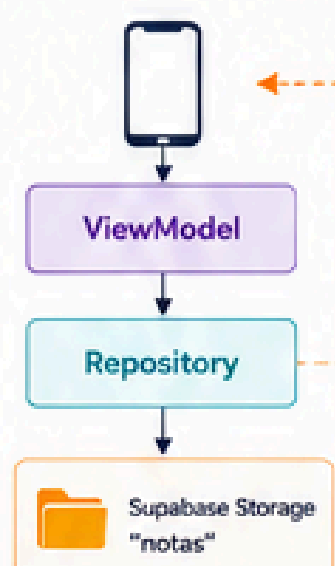
- Pantalla de estadísticas**
Observa AcademicStatsViewModel

- Pantalla de pasos**
Observa PedometerViewModel

- Otras pantallas**
(Login, Perfil, Mapas, etc.)
Observan otros ViewModels

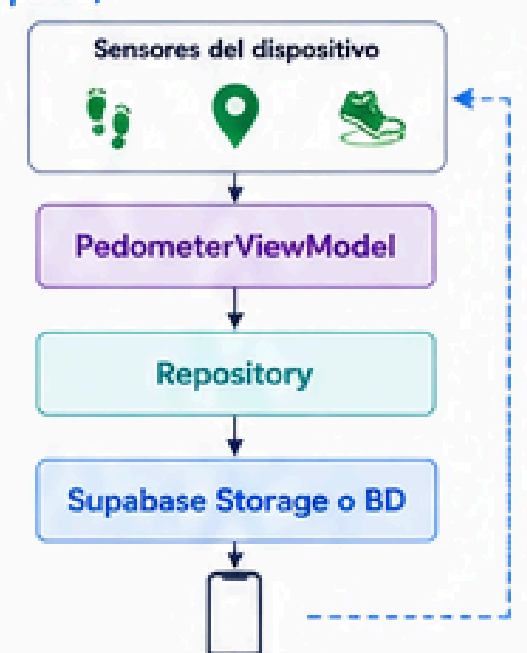
5 FLUJO COMPLETO DE DATOS - EJEMPLO 1 (Editar notas)

- 1 User edita nota en pantalla
- 2 AcademicStatsViewModel.guardarMaterias (materias actualizadas)
- 3 Función recalcular() → recalcula todos los promedios
- 4 AcademicRepository.saveEstadísticas()
- 5 Supabase Storage "notas" → guarda JSON actualizado
- 6 StateFlow se actualiza
- 7 UI muestra nuevos promedios



6 FLUJO COMPLETO DE DATOS - EJEMPLO 2 (Conteo de pasos)

- 1 Dispositivo detecta paso
- 2 [SENSOR STEP DETECTOR] → PedometerViewModel.stepListener
- 3 ¿Usuario dentro del campus? → Verifica con (GPS/LOCATION) + geojson
 - NO → Continúa contando
 - SI → (actualiza estado)
- 4 _stepsToday.value (actualiza estado)
- 5 ¿Ha acumulado 50 pasos? → Continúa contando (NO) / SI → PedometerRepository.saveStats()
- 6 PedometerRepository.saveStats() (guarda registro de pasos/distancia)
- 7 Supabase Storage o BD → persiste datos
- 8 StateFlow se actualiza
- 9 UI muestra pasos y distancia actualizada



LEYENDA: Acceso a datos (Repository) Lógica y estados (ViewModel) Estructuras de datos (Model) Presentación (UI) Flujo de datos Observación (StateFlow) Conexión externa

Sensores

luminosidad

Pedometro

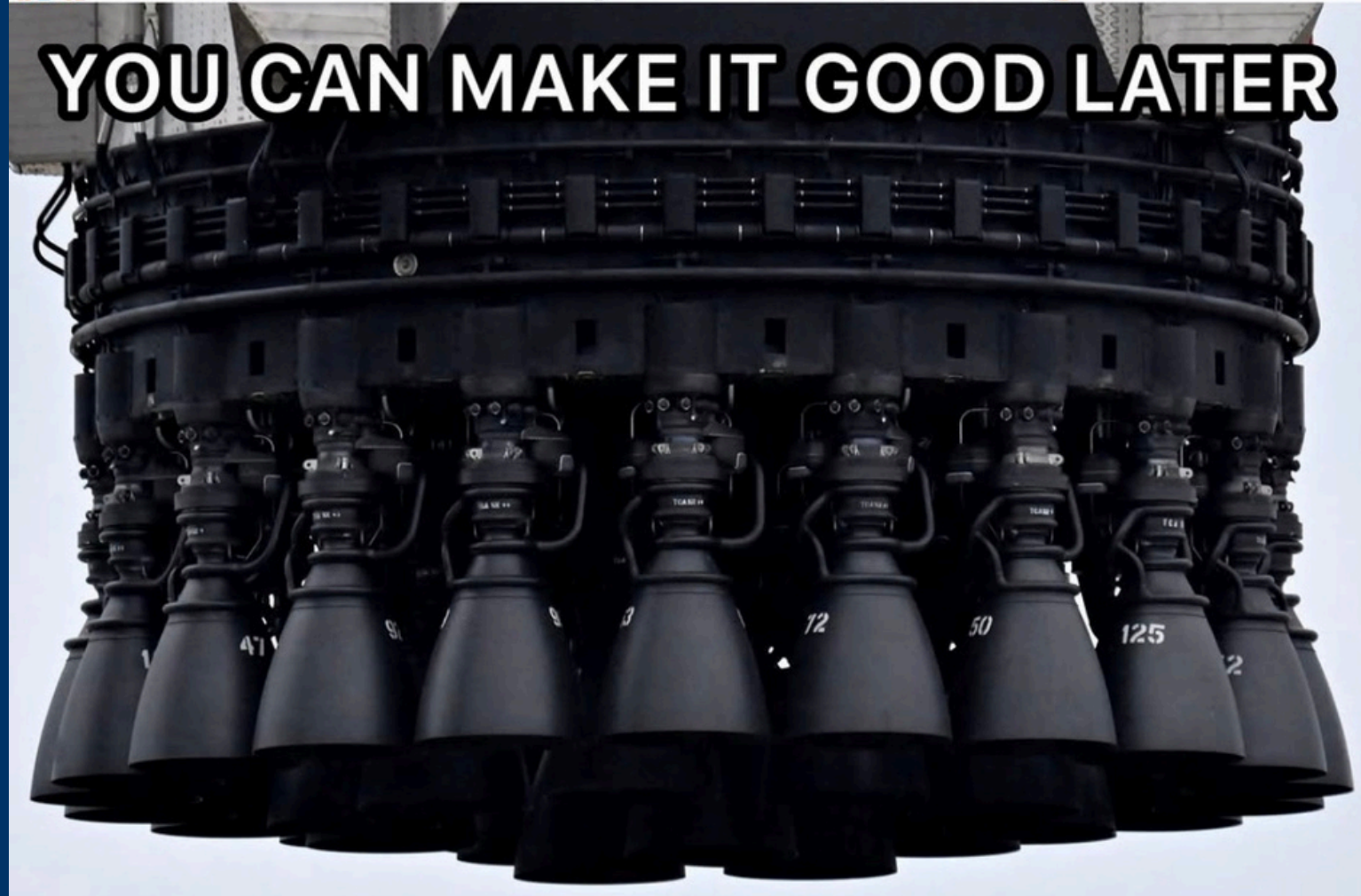
Acelerometro

Giroscopio

JUST MAKE IT EXIST FIRST



YOU CAN MAKE IT GOOD LATER



Conclusiones

Demo